Content Addressable Memory (CAM) Verilog Implementation

Arpith Jacob Alex
Electronics and VLSI Design Engineering
Saintgits College of Engineering
arpithalex07@gmail.com

CHIPTHINK VLSI Hackathon – February 2026
Maven Silicon — Christ College of Engineering

Abstract

This report presents a parameterized Content Addressable Memory (CAM) implementation in Verilog, developed as part of the CHIPTHINK VLSI Hackathon. Unlike traditional RAM that searches by address, CAM enables content-based searching — a critical feature in network routers, CPU caches, and database systems. The design includes write operations, content-based search with priority encoding, entry deletion, and status monitoring. This implementation was completed within a 24-hour timeframe without AI assistance and later refined based on industry feedback.

1 Introduction

1.1 What is Content Addressable Memory?

Content Addressable Memory (CAM) is a specialized memory architecture that inverts the traditional memory access paradigm:

- **Traditional RAM:** Address → Data
- **CAM:** Data → Address

When given a search key, CAM returns the address(es) where that data is stored, enabling constant-time lookups regardless of memory size.

1.2 Applications

CAM is extensively used in:

- **Network Switches:** MAC address lookup tables
- **Routers:** IP routing table matching
- **CPU Caches:** Translation Lookaside Buffers (TLB)
- **Database Systems:** Fast key-based indexing

2 Design Architecture

2.1 Key Features

The implementation includes:

- Fully parameterized depth, width, and address size
- Valid-bit tracking for each memory entry
- Priority-based matching (lowest index returned on duplicates)
- Entry deletion/invalidation
- Synchronous reset
- Full and empty status flags

2.2 Parameters

Parameter	Default	Description
DEPTH	16	Number of memory entries
WIDTH	8	Data word width (bits)
ADDR_WIDTH	4	Address width ($\geq \log_2(DEPTH)$)

3 Implementation Details

3.1 Memory Structure

The CAM consists of two primary components:

```
// Main memory storage
reg [WIDTH-1:0] memory [0:DEPTH-1];

// Valid bit array - tracks occupied entries
reg valid [0:DEPTH-1];
```

Listing 1: Memory Array Declaration

Each memory location has an associated valid bit, allowing efficient tracking of occupied versus empty slots.

3.2 Write Operation

```
always @(posedge clk) begin
    if (rst) begin
        for (i = 0; i < DEPTH; i = i + 1) begin
            valid[i] <= 1'b0;
        end
    end
    else if (we && !full) begin
        memory[waddr] <= din;
        valid[waddr] <= 1'b1;
    end
    else if (delete_en) begin
        valid[waddr] <= 1'b0;
    end
end
```

Listing 2: Write Logic

Write operations check the full flag before storing data and automatically set the valid bit.

3.3 Search Logic with Priority Encoding

The search operation implements priority encoding to handle duplicate entries:

```
always @(posedge clk) begin
    if (rst) begin
        match <= 1'b0;
        match_index <= {ADDR_WIDTH{1'b0}};
        busy <= 1'b0;
    end
    else if (search_en) begin
        match <= 1'b0;
        busy <= 1'b1;

        // Priority encoder: return lowest index on match
        for (i = 0; i < DEPTH; i = i + 1) begin
            if (valid[i] && memory[i] == search_key) begin
                if (!match) begin  // First match wins
                    match <= 1'b1;
                    match_index <= i;
                end
            end
        end
    end
    else begin
        busy <= 1'b0;
    end
end
```

Listing 3: Content-Based Search

The priority encoder ensures deterministic behavior when multiple entries contain the same value.

3.4 Status Flag Generation

```
integer j;
always @(*) begin
    valid_count = 0;
    for (j = 0; j < DEPTH; j = j + 1) begin
        if (valid[j]) valid_count = valid_count + 1;
    end
end

assign full = (valid_count == DEPTH);
assign empty = (valid_count == 0);
```

Listing 4: Full and Empty Detection

These combinational flags provide real-time memory occupancy status.

4 Verification

4.1 Testbench Coverage

The testbench validates:

- **Reset Behavior:** Clears all valid bits and status flags
- **Write Operations:** Correctly stores data with valid bit set
- **Search Hit:** Returns correct index for existing values
- **Search Miss:** No false positives for absent values
- **Priority Encoding:** Lowest index returned for duplicates
- **Deletion:** Entry invalidated, next copy found in subsequent searches
- **Overwrite:** Old value replaced, new value searchable
- **Full Flag:** Asserted when all entries occupied

4.2 Simulation Results

[Simulation waveforms would be included here showing key test scenarios]

5 Design Trade-offs and Optimizations

5.1 Hackathon Constraints

During the 24-hour hackathon, several design decisions were made under time pressure:

- **Linear Search:** Implemented for simplicity; scalable to parallel comparators
- **Single Clock Cycle Search:** Trade-off between latency and logic complexity
- **Priority Encoding:** First-match approach chosen for determinism

5.2 Post-Hackathon Improvements

Based on industry feedback from Maven Silicon judges:

- Enhanced testbench with comprehensive corner case coverage
- Added detailed inline comments for code clarity
- Improved documentation with timing diagrams
- Optimized valid-bit counting logic

6 Conclusion

This CAM implementation demonstrates fundamental principles of content-addressable memory design while adhering to industry best practices. The 24-hour hackathon constraint forced focus on core functionality and robust logic design rather than feature creep.

Key learnings include the importance of:

- Systematic verification before adding complexity
- Clean parameter-based design for reusability
- Understanding hardware implications of RTL choices
- Comprehensive documentation for maintainability

Acknowledgments

Developed during the CHIPTHINK VLSI Hackathon organized by Maven Silicon at TECHLETICS 4.0, Christ College of Engineering. Special thanks to the Maven Silicon judges for valuable industry feedback.

Repository: `https://github.com/Arpith173/CAM-Content-Addressable-Memory`